

Guide de Déploiement Gestion des Mutations

Installation sur Windows

TECHNOLOGIE

Laravel 12 · PHP 8.2+

BASE DE DONNÉES

SQLite

VERSION DOC

Juin 2026

PLATEFORME CIBLE

Windows 10 / 11

Table des matières

1. Prérequis logiciels 3

1.1 PHP 8.2+ 3

1.2 Composer 3

1.3 Node.js & npm 3

1.4 Git (optionnel) 3

2. Récupération du projet 4

3. Configuration de l'environnement 4

4. Installation des dépendances 5

5. Initialisation de la base de données 5

6. Compilation des assets frontend 6

7. Démarrage de l'application 6

8. Comptes utilisateurs par défaut 7

9. Extensions PHP requises 7

10. Structure du projet 8

11. Résolution des problèmes courants 9

12. Récapitulatif rapide (Quick Start) 10

1. Prérequis logiciels

Avant toute installation, vérifiez que les logiciels suivants sont présents sur la machine Windows cible. Tous sont gratuits et disponibles publiquement.

1.1 PHP 8.2 ou supérieur

L'application nécessite **PHP 8.2+** avec plusieurs extensions activées. La solution recommandée sur Windows est d'utiliser **Laragon**, qui embarque PHP, un serveur web Apache et SQLite en un seul installateur.

? Solution recommandée : Laragon

Télécharger Laragon Full (inclut PHP 8.2, Apache, MySQL, SQLite) :

<https://laragon.org/download/>

Choisir **laragon-full.exe** (version > 6.0).

? Alternative : PHP standalone

Télécharger PHP 8.2 Thread Safe depuis <https://windows.php.net/download/> et ajouter le dossier PHP au **PATH Windows**.

1.2 Composer (gestionnaire de paquets PHP)

Composer gère toutes les bibliothèques PHP du projet (Laravel, DomPDF, Excel, QR Code...).

- Télécharger **Composer-Setup.exe** depuis <https://getcomposer.org/download/>
- Lors de l'installation, l'assistant détecte automatiquement le chemin vers `php.exe`
- Vérification : ouvrir un terminal et taper `composer --version`

1.3 Node.js & npm

Requis pour compiler les assets CSS/JS (Vite + Tailwind CSS). Version minimale : **Node.js 18**.

- Télécharger l'installateur LTS depuis <https://nodejs.org/>
- npm est inclus avec Node.js
- Vérification : `node --version` et `npm --version`

1.4 Git (optionnel)

Git est recommandé pour cloner le projet depuis un dépôt. Si le projet est fourni en archive ZIP, Git n'est pas obligatoire.

- Télécharger depuis <https://git-scm.com/download/win>

? Résumé des versions minimales requises

Logiciel	Version minimale	Recommandé
PHP	8.2	8.2.x ou 8.3.x
Composer	2.0	2.7+
Node.js	18.0	20 LTS
npm	9.0	10+

2. Récupération du projet

Le projet peut être obtenu via Git ou via une archive ZIP fournie par l'équipe.

Option A — Via Git (recommandé)

```
# Ouvrir un terminal (cmd ou PowerShell) dans le dossier cible # Par exemple : C:\projets\ git clone  
<URL_DU_DEPOT> gestion-mutations cd gestion-mutations
```

Option B — Via archive ZIP

1

Extraire l'archive

Décompresser le fichier `gestion-mutations.zip` dans un dossier, par exemple `C:\projets\gestion-mutations\`.

2

Ouvrir un terminal dans ce dossier

Dans l'Explorateur Windows, naviguer vers le dossier extrait, puis *Maj + clic droit ? Ouvrir PowerShell ici*.

? Important — Chemin sans espaces

Éviter les chemins avec espaces ou caractères accentués (ex : `C:\Mes Documents\`). Préférer `C:\projets\` ou `C:\www` pour éviter des erreurs PHP/Composer.

3. Configuration de l'environnement

L'application utilise un fichier `.env` pour toutes les variables d'environnement (clé app, base de données, mail...).

3.1 Créer le fichier .env

```
# Dans le dossier du projet : copy .env.example .env
```

3.2 Modifier les variables essentielles

Ouvrir `.env` avec un éditeur de texte (Notepad++, VS Code) et ajuster :

```
APP_NAME="Gestion des Mutations" APP_ENV=local APP_DEBUG=true APP_URL=http://localhost:8000 # Base de données SQLite (aucune configuration serveur requise) DB_CONNECTION=sqlite # Pour la session et le cache SESSION_DRIVER=database CACHE_STORE=database QUEUE_CONNECTION=database
```

? À propos de SQLite

Le projet utilise **SQLite** par défaut — aucun serveur de base de données (MySQL, PostgreSQL) n'est requis. Le fichier de base de données se crée automatiquement dans `database/database.sqlite`.

4. Installation des dépendances

Toutes les bibliothèques PHP et Node.js doivent être installées avant de lancer l'application.

4.1 Dépendances PHP (Composer)

Dans le terminal, depuis le dossier du projet :

```
composer install
```

Cette commande télécharge automatiquement :

- **Laravel 12** — framework principal
- **barryvdh/laravel-dompdf** — génération de PDF pour les notifications
- **maatwebsite/excel** — import/export des fichiers Excel
- **simplesoftwareio/simple-qrcode** — génération des QR codes de vérification
- **spatie/laravel-permission** — gestion des rôles (admin, receveur, gestionnaire, opérateur)

? Durée

L'installation dure 2 à 5 minutes selon la vitesse de connexion. Ne pas interrompre la commande.

4.2 Génération de la clé d'application

```
php artisan key:generate
```

Cette commande crée la clé `APP_KEY` dans le fichier `.env`. Elle est indispensable pour le chiffrement des sessions.

5. Initialisation de la base de données

La base de données doit être créée, les tables générées et les données de base insérées.

5.1 Créer le fichier SQLite

```
# Créer le fichier SQLite vide (PowerShell) New-Item -Path database\database.sqlite -ItemType File #  
cmd : type nul > database\database.sqlite
```


5.2 Exécuter les migrations

Les migrations créent toutes les tables nécessaires (communes, projets, parcelles, propriétaires, mutations, imports, templates...) :

```
php artisan migrate
```

5.3 Insérer les données initiales

```
php artisan db:seed
```

Cette commande insère :

- Les **rôles et permissions** (admin, receveur, gestionnaire, opérateur)
- Le **template de document** de notification d'attribution
- Des **données de test** (communes, projets, parcelles, mutations d'exemple)
- Les **comptes utilisateurs** de démonstration

? Commande tout-en-un (migrations + seed)

```
php artisan migrate:fresh --seed
```

Réinitialise complètement la base et insère les données de départ. Idéal pour un premier déploiement.

6. Compilation des assets frontend

Les fichiers CSS (Tailwind) et JS doivent être compilés avant que l'interface soit utilisable.

6.1 Installer les dépendances Node.js

```
npm install
```

6.2 Compiler pour la production

```
npm run build
```

Cette commande génère les fichiers optimisés dans `public/build/`. À utiliser pour un déploiement stable.

? Mode développement (avec rechargement automatique)

```
npm run dev
```

À utiliser uniquement en phase de développement. Laisser ce terminal ouvert en parallèle du serveur PHP.

6.3 Permissions du dossier storage

Sur Windows, s'assurer que le dossier `storage\` est accessible en écriture par PHP :

```
php artisan storage:link
```

Crée le lien symbolique `public/storage` ? `storage/app/public`.

7. Démarrage de l'application

Deux modes de démarrage sont disponibles selon l'usage.

Mode simple — Serveur intégré PHP

Pour un usage local sans configuration serveur web :

L'application est accessible sur : **http://localhost:8000**

? URL d'accès

Ouvrir un navigateur (Chrome, Edge) et aller sur : **http://127.0.0.1:8000**

Mode complet — Tout en une commande

Lance simultanément le serveur PHP, la queue, les logs et Vite :

```
composer run dev
```

Ce mode est recommandé pour le développement actif. Nécessite que `concurrently` soit installé (inclus dans `npm`

? Ne pas fermer le terminal

Le serveur PHP s'arrête si le terminal est fermé. Pour un usage permanent, configurer Laragon pour servir l'application (voir s 11).

8. Comptes utilisateurs par défaut

Après le `db:seed`, les comptes suivants sont disponibles. Le mot de passe initial est **password** pour tous.

Email	Nom	Rôle	Accès
<code>admin@domaines.sn</code>	Administrateur	admin	Accès total — gestion utilisateurs, templates
<code>receveur@domaines.sn</code>	Mamadou DIALLO	receveur	Validation mutations, rapports, PDF
<code>gestionnaire@domaines.sn</code>	Abdoulaye NDIAYE	gestionnaire	Imports, gestion parcelles et mutations
<code>opérateur@domaines.sn</code>	Fatou SARR	opérateur	Saisie et consultation uniquement

? Sécurité — Production

Changer impérativement les mots de passe avant toute utilisation en production. Désactiver le compte admin de test après création d'un vrai compte administrateur.

9. Extensions PHP requises

Vérifier que les extensions suivantes sont activées dans le fichier `php.ini`. Avec Laragon, la plupart sont déjà activées.

Extension	Usage	Statut Laragon
<code>pdo_sqlite</code>	Base de données SQLite	Activée
<code>sqlite3</code>	Support SQLite natif	Activée
<code>gd</code>	Génération des QR codes (images PNG)	Activée
<code>mbstring</code>	Gestion des chaînes multi-octets (accents)	Activée
<code>openssl</code>	Chiffrement, sessions sécurisées	Activée
<code>fileinfo</code>	Détection du type MIME des fichiers uploadés	Activée
<code>zip</code>	Import/export Excel (maatwebsite)	Vérifier
<code>xml</code>	Traitement des fichiers Excel (format OOXML)	Vérifier

intl

Formatage des dates en français

Vérifier

Activer une extension dans php.ini

Localiser le fichier `php.ini` (souvent dans `C:\laragon\bin\php\php-8.2.x\php.ini`) et décommenter la ligne correspondante :

```
# Avant (commenté) : ;extension=gd # Après (activé) : extension=gd
```

Redémarrer le serveur PHP après toute modification de `php.ini`.

10. Structure du projet

Description des dossiers et fichiers importants à connaître pour l'administration et la personnalisation.

Chemin	Description
app/Http/Controllers/	Contrôleurs (logique métier : mutations, imports, rapports...)
app/Models/	Modèles Eloquent (Mutation, Parcelle, Proprietaire, Import...)
database/migrations/	Structure des tables de la base de données
database/seeder/	Données initiales (rôles, template, données de test)
database/database.sqlite	Fichier de base de données SQLite
resources/views/	Templates Blade (interface HTML de l'application)
public/images/	Images statiques : logo DGID, en-tête des documents PDF
public/build/	Assets compilés (CSS/JS) — généré par <code>npm run build</code>
storage/app/	Fichiers générés (QR codes temporaires, exports)
storage/logs/	Journaux de l'application (<code>laravel.log</code>)
.env	Configuration locale (clés, base de données, mail)
routes/web.php	Définition de toutes les routes HTTP

Modules fonctionnels

Module	Description	URL
Tableau de bord	Statistiques globales et dernières mutations	/dashboard
Mutations	Liste, création, validation et PDF des mutations	/mutations
Imports Excel	Import en masse via fichier Excel	/imports
Parcelles	Consultation et gestion des lots	/parcelles
Projets	Gestion des lotissements par commune	/projets

Module	Description	URL
Rapports	Rapports filtrés exportables en PDF	/rapports
Vérification QR	Scan et vérification des documents	/scanner
Templates	Éditeur de modèle de notification (HTML)	/templates
Annulations	Demandes et approbations d'annulation	/annulations
Administration	Export/import de la base de données	/admin/export-import

Images personnalisables

Le dossier `public/images/` contient les images utilisées dans les documents PDF générés. Pour personnaliser l'en-tête et les notifications :

- Remplacer `public/images/entete_dgid.png` par l'en-tête officiel souhaité
- Maintenir les mêmes dimensions (largeur ~800px recommandée)
- Le logo `public/images/dgid-logo.png` est utilisé dans la barre latérale

11. Résolution des problèmes courants

Liste des erreurs fréquentes rencontrées lors du déploiement sur Windows et leur solution.

Erreur : `Could not open input file: artisan`

Le terminal n'est pas dans le bon dossier. Vérifier avec :

```
dir artisan # doit afficher le fichier artisan
```

Naviguer dans le dossier du projet : `cd C:\projets\gestion-mutations`

Erreur : `Class "PDO" not found` OU `pdo_sqlite`

L'extension PDO SQLite n'est pas activée. Dans `php.ini`, décommenter :

```
extension=pdo_sqlite extension=sqlite3
```

Erreur : `The stream or file ... could not be opened`

Le dossier `storage\` n'est pas accessible en écriture. Dans PowerShell en admin :

```
icacls storage /grant Users:F /T icacls bootstrap\cache /grant Users:F /T
```

Erreur : `Vite manifest not found`

Les assets n'ont pas été compilés. Exécuter :

```
npm install npm run build
```

Erreur : `php_gd2.dll - module not found`

L'extension GD est requise pour les QR codes. Activer dans `php.ini` :

```
extension=gd
```

Erreur : `No application encryption key`

La clé d'application n'a pas été générée :

```
php artisan key:generate
```

Page blanche ou erreur 500

Consulter le fichier de log pour connaître la cause exacte :

```
# Afficher les dernières erreurs : type storage\logs\laravel.log | more
```

Vérifier aussi que `APP_DEBUG=true` est bien défini dans `.env` pour afficher le détail de l'erreur dans le navigateur.

La session expire immédiatement

La table de sessions n'a pas été créée. Vérifier :

```
php artisan migrate php artisan cache:clear php artisan config:clear
```

Caractères accentués incorrects dans les PDF

L'extension `intl` est nécessaire pour le formatage des dates en français. Activer dans `php.ini` :

```
extension=intl
```


? Support technique

En cas de problème non résolu par ce guide, consulter le fichier `storage/logs/laravel.log` et transmettre les dernières d'erreur au responsable technique du projet.

? Exporter en PDF (Ctrl+P ? Enregistrer en PDF)

Dans la boîte de dialogue d'impression : Destination ? **Enregistrer au format PDF** · Format ? **A4** · Marges ? **Aucune**